MAKERERE UNIVERSITY
COLLEGE OF COMPUTING AND INFORMATICS TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE
COURSE: CLOUD COMPUTING
ASSIGNMENT: LAB 1
GROUP: DAZ

Mwizerwa Timothy	2400724330	24/U/24330/PS
Kinda Kelsey Naluwafu	2400705862	24/U/05862/PS
Kato Adrian Mugerwa	2400725909	24/U/08847/PS
Naluyima Pauline Oliver	2400708847	24/U/08847/PS
Aroba Joseph	2400703646	24/U/03646/EVE

1. What is the bandwidth between the client and server machines on the bare metal setup? (2 Marks)
   - **9.4Gbits/sec**

```
[  5]   8.00-9.00   sec  1.09 GBytes  9.41 Gbits/sec    0   1.41 MBytes

[  5]   9.00-10.00  sec  1.09 GBytes  9.40 Gbits/sec    0   1.41 MBytes

- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec  10.9 GBytes  9.40 Gbits/sec   12             sender
[  5]   0.00-10.00  sec  10.9 GBytes  9.40 Gbits/sec                  receiv
er

iperf Done.
csc26daz@client0:~$
```

2. What is the average latency for the requests sent by wrk2? (2 Marks)
   - **1.212ms**

```
      14.095     0.999945        19994     18204.44
      15.735     0.999951        19995     20480.00
      15.735     1.000000        19995          inf
#[Mean    =        1.212, StdDeviation   =        0.477]
#[Max     =       15.728, Total count    =        19995]
#[Buckets =           27, SubBuckets     =         2048]
----------------------------------------------------------
  20005 requests in 10.00s, 32.30MB read
Requests/sec:   2000.71
csc26daz@client0:/local/repository/csc2202-cloud-project/DeathStarBench/hote
csc26daz@client0:/local/repository/csc2202-cloud-project/DeathStarBench/hote
lReservation$
```

3. Explain the difference between open-loop and closed-loop load generators. Which one is wrk2? Give one advantage and one disadvantage of each type of load generator? (4 Marks)
   - An **open-loop** generator sends requests on a fixed schedule regardless of whether previous responses have arrived whereas a **closed-loop** generator only sends a new request after receiving a response from the previous one.
   - wrk2 is open-loop.
   - **Open-loop**
     - ADVANTAGE: mimics real traffic
     - DISADVANTAGE: can overwhelm the server
   - **Closed-loop**
     - ADVANTAGE: it's more controlled and stable
     - DISADVANTAGE: it does not reflect real-world traffic patterns

4. How many replicas of each service are running in the hotel reservation application? Why do you think the application is designed this way?
   - Most services run 1/1 replica except hotelreservation_frontend which runs 3/3
   - REASON: For load balancing and high availability since frontend handles all incoming traffic

```
csc26daz@server0:/local/repository/csc2202-cloud-project$ sudo docker service ls
ID             NAME                                   MODE         REPLICAS   IMAGE                                                    PORTS
swv3imc067ci   hotelreservation_consul                replicated   1/1        consul:1.15                                              *:8300->8300/tc
p, *:8400->8400/tcp, *:8500->8500/tcp, *:8600->53/udp
o9aetw154h09   hotelreservation_frontend              replicated   3/3        igorrudyk1/hotel_reserv_frontend_single_node:latest      *:5000->5000/tc
p
pifydw0f35y3   hotelreservation_geo                   replicated   1/1        igorrudyk1/hotel_reserv_geo_single_node:latest
ktwzxyjia3kz   hotelreservation_jaeger                replicated   1/1        jaegertracing/all-in-one:latest                          *:5778->5778/tc
p, *:14268->14268/tcp, *:16686->16686/tcp, *:30000->14269/tcp, *:30001->14267/tcp, *:5775->5775/udp, *:6831-6832->6831-6832/udp
gxmle3ehg584   hotelreservation_memcached-profile     replicated   1/1        memcached:latest
pbrjc5t4rezn   hotelreservation_memcached-rate        replicated   1/1        memcached:latest
xqe2vxvop1or   hotelreservation_memcached-reserve     replicated   1/1        memcached:latest
4fwi08be7csq   hotelreservation_mongodb-geo           replicated   1/1        mongo:4.4.6
kupx4io04r7x   hotelreservation_mongodb-profile       replicated   1/1        mongo:4.4.6
ny0ywedotxku   hotelreservation_mongodb-rate          replicated   1/1        mongo:4.4.6
lemlhkwjouzu   hotelreservation_mongodb-recommendation replicated   1/1        mongo:4.4.6
xdeqcof9ezw7   hotelreservation_mongodb-reservation   replicated   1/1        mongo:4.4.6
zj6p1f0f26f8   hotelreservation_mongodb-user          replicated   1/1        mongo:4.4.6
g4zqm6shn1ru   hotelreservation_profile               replicated   1/1        igorrudyk1/hotel_reserv_profile_single_node:latest
44vg2yzkxgd2   hotelreservation_rate                  replicated   1/1        igorrudyk1/hotel_reserv_rate_single_node:latest
qinpnnkr1zjp   hotelreservation_recommendation        replicated   1/1        igorrudyk1/hotel_reserv_recommendation_single_node:latest
chvavdfw711k   hotelreservation_reservation           replicated   1/1        igorrudyk1/hotel_reserv_reserve_single_node:latest
ul5olnbjahfn   hotelreservation_search                replicated   1/1        igorrudyk1/hotel_reserv_search_single_node:latest
kjf7jrwpvhld   hotelreservation_user                  replicated   1/1        igorrudyk1/hotel_reserv_user_single_node:latest
csc26daz@server0:/local/repository/csc2202-cloud-project$
```

5. In the Docker Swarm deployment of the hotel reservation application, which services are running on server0 and which services are running on server1?

```
csc26daz@server0:~$ for service in $(sudo docker service ls --format "{{.Name}}"); do echo "=== $service ==="; sudo docker service
ps $service --format "{{.Name}} {{.Node}}" --filter "desired-state=running"; done
=== hotelreservation_consul ===
hotelreservation_consul.1 server0.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_frontend ===
hotelreservation_frontend.1  server1.daz-lab1.cranecloud-pg0.apt.emulab.net
hotelreservation_frontend.2  server1.daz-lab1.cranecloud-pg0.apt.emulab.net
hotelreservation_frontend.3  server0.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_geo ===
hotelreservation_geo.1 server1.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_jaeger ===
hotelreservation_jaeger.1  server0.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_memcached-profile ===
hotelreservation_memcached-profile.1 server0.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_memcached-rate ===
hotelreservation_memcached-rate.1 server1.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_memcached-reserve ===
hotelreservation_memcached-reserve.1 server0.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_mongodb-geo ===
hotelreservation_mongodb-geo.1 server1.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_mongodb-profile ===
hotelreservation_mongodb-profile.1 server0.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_mongodb-rate ===
hotelreservation_mongodb-rate.1 server1.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_mongodb-recommendation ===
hotelreservation_mongodb-recommendation.1 server0.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_mongodb-reservation ===
hotelreservation_mongodb-reservation.1 server1.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_mongodb-user ===
hotelreservation_mongodb-user.1 server0.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_profile ===
hotelreservation_profile.1 server0.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_rate ===
hotelreservation_rate.1  server1.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_recommendation ===
hotelreservation_recommendation.1  server1.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_reservation ===
hotelreservation_reservation.1  server0.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_search ===
hotelreservation_search.1 server1.daz-lab1.cranecloud-pg0.apt.emulab.net
=== hotelreservation_user ===
hotelreservation_user.1  server0.daz-lab1.cranecloud-pg0.apt.emulab.net
csc26daz@server0:~$
```

   - **server0:** consul, frontend.3, jaeger, memcached-profile, memcached-reserve, mongodb-profile, mongodb-recommendation, mongodb-user, profile, reservation, user

- **server1:** frontend.1, frontend.2, geo, memcached-rate, mongodb-geo, mongodb-rate, mongodb-reservation, rate, recommendation, search
- Docker Swarm distributed them automatically to balance the load across both servers.

6. Document your lessons learned and any difficulties you encountered while doing this Lab. (3 Marks)
   - One of the main difficulties encountered during this lab was setting up SSH agent forwarding on Windows. The start_docker.sh script requires SSH access from server0 to the other nodes in the experiment, which depends on agent forwarding being active. However, Windows does not run the SSH agent automatically, and despite starting it manually via PowerShell and using the -A flag when connecting, the agent was not being forwarded to server0. This was resolved by manually copying the private key onto server0, which allowed the script to authenticate successfully.

```
csc26daz@server0:/local/repository/csc2202-cloud-project$ ./start_docker.sh
n_servers=2
n_clients=1
Warning: Permanently added 'server0,10.10.1.1' (ECDSA) to the list of known hosts.
csc26daz@server0: Permission denied (publickey).
Warning: Permanently added 'server1,10.10.1.2' (ECDSA) to the list of known hosts.
csc26daz@server1: Permission denied (publickey).
Error: docker is not running on server0

csc26daz@server0:/local/repository/csc2202-cloud-project$ ./start_docker.sh
n_servers=2
n_clients=1
csc26daz@server0: Permission denied (publickey).
csc26daz@server1: Permission denied (publickey).
Error: docker is not running on server0
csc26daz@server0:/local/repository/csc2202-cloud-project$
```

   - Another challenge was installing the iperf3 and wrk2 dependencies on the client machine. The apt package list was outdated, requiring a sudo apt update before packages could be installed.
   - On the positive side, the deployment of DeathStarBench using Docker Swarm was straightforward once the SSH issue was resolved. The docker stack deploy command automatically distributed services across server0 and server1, demonstrating how Docker Swarm handles workload distribution without manual configuration. Running wrk2 and observing the latency statistics gave a practical understanding of how open-loop load generators work and how microservices handle concurrent requests.
   - Overall, the lab provided valuable hands-on experience with cloud infrastructure, distributed systems, and network benchmarking tools.